ARIEMEDI

精准导航　智引未来

瑞瞳 SDK 开发说明文档

Windows C++版

版本号 V4.8.0

型号规格 RUITONG SDK

艾瑞迈迪医疗科技（北京）有限公司

目录

1. 简介 5
1.1 产品概述 5
1.2 环境要求 5
1.2.1 系统要求 5
1.2.2 开发环境与编译语言 5
2. SDK 接口说明 5
2.1 设备扫描接口 5
2.1.1 getDeviceInfo 5
2.1.2 updateDeviceInfo 5
2.2 设备连接接口 6
2.2.1 connect 6
2.2.2 disconnect 6
2.2.3 getConnectedDeviceInfo 6
2.2.4 printAPIandFirmwareVersion 7
2.3 器械设置接口 7
2.3.1 generateAROM 7
2.3.2 convert2AROM 8
2.3.3 loadPassiveToolAROM 8
2.3.4 loadPassiveToolNAROM 9
2.3.5 loadActiveToolAROM 9
2.3.6 getToolStorage 10
2.3.7 getTrackToolsNum 10
2.3.8 clearToolStorage 11
2.3.9 updateToolStorage 11
2.3.10 getToolCalibrationData 11
2.4 数据传输接口 11
2.4.1 setTrackingDataTransmissionType 12
2.4.2 setTrackingDataTransformation 12
2.4.3 startTracking 12
2.4.4 stopTracking 13
2.4.5 startImaging 13
2.4.6 stopImaging 13
2.4.7 startVideoMonitor 14
2.4.8 stopVideoMonitor 14
2.4.9 setImagingMode 14
2.4.10 reconstructPointCloud 15
2.4.11 evaluateReconstruction 15
2.4.12 setMaterial 15
2.4.13 trackingUpdate 16
2.4.14 monitoringUpdate 16
2.4.15 getAllMarkers 16
2.4.16 getIFImageSize 17

2.4.17 getVLImageSize.....17
2.4.18 setVLImageResolution.....17
2.4.19 getLeftImagingData.....18
2.4.20 getRightImagingData.....18
2.4.21 getVideoMonitorData.....18
2.4.22 getReconstructionData.....18
2.4.23 getTrackingData.....19
2.4.24 getUnMatchedMarkers.....19
2.4.25 getSystemTime.....19
2.4.26 getReconstructParameters.....20
2.5 状态监测接口.....20
2.5.1 getGravityVector.....20
2.5.2 getConnectionStatus.....20
2.5.3 getTransmissionStatus.....21
2.5.4 getReconstructionStatus.....21
2.5.5 getSystemAlert.....21
2.6 常用工具接口.....22
2.6.1 pivotTipCalibration.....22
2.6.2 fixedTipCalibration.....23
2.6.3 fixedDirCalibration.....23
2.6.4 setIFLuminance.....24
2.6.5 setIFContrast.....24
2.6.6 setAreaDisplay.....25
2.6.7 setAreaHidden.....25
2.6.8 trackingDataProjection.....25
2.6.9 setReferenceTool.....26
2.6.10 getTrackingVolume.....26
2.6.11 markerInOutTrackingVolume.....27
2.6.12 beep.....27
2.6.13 laser.....27
2.6.14 laserOn.....27
2.6.15 getNetAdaptorInfo.....28
2.6.16 setDeviceConnectionMode.....28
2.6.17 getLog.....29
2.6.18 saveLog.....29
2.7 点云重建接口.....29
2.7.1 initializeParameters.....29
2.7.2 setReconstructData.....30
2.7.3 getPointCloud.....30
2.7.4 getTextureImageMat.....30
2.7.5 getTextureImageUChar.....30
2.7.6 getDepthImage.....31
2.7.7 getDepthImageSize.....31
2.7.8 setDefaultColor.....31

2.7.9 project2DImagePosition 32
2.7.10 project2DImageArea 32
2.7.11 project2DImageRectArea 32
2.7.12 project2DImageCircleArea 33
2.7.13 savePLY 33
2.8 增强现实融合显示接口 34
2.8.1 initialization 34
2.8.2 getARprojection 34
3. SDK 重要数据结构 35
3.1 器械标定信息数据结构 **ToolCalibrationData** 35
3.2 跟踪信息数据结构 **ToolTrackingData** 35
3.3 标志物数据结构 **MarkerPosition** 36
3.4 器械平面数据结构 **MarkerPlane** 36
3.5 跟踪位姿信息数据结构 **Transformation** 36
3.6 点数据结构 **PointInfo** 36
3.7 点云数据结构 **PointCloudInfo** 37
3.8 网卡信息数据结构 **NetAdaptorInfo** 37
3.9 连接设备信息数据结构 **DeviceInfo** 37
4. SDK 重要枚举类型的命名空间 37
4.1 设备类型 **DeviceType** 37
4.2 器械标定文件生成状态 **GenerationStatus** 38
4.3 设备传输数据类型 **TransmissionStatus** 38
4.4 设备跟踪数据传输类型 **TransmissionType** 39
4.5 设备连接状态 **ConnectionStatus** 39
4.6 设备警告信息 **DeviceAlert** 39
4.7 器械标定状态 **CalibrationAlert** 40
4.8 器械跟踪状态 **TransformationStatus** 40
5. Demo 使用指南 40
5.1 连接准备 40
5.2 文件目录 41
5.3 使用流程 41
5.4 工程配置 41
5.5 调用流程 41
6. 常见问题 42

1. 简介

1.1 产品概述

适用于瑞瞳系列产品（瑞瞳 SE、瑞瞳 LITE、瑞瞳 MAX、瑞瞳 MAXV、瑞瞳 PRO）的 SDK 工具包，包含瑞瞳的连接与初始化、标志物定位、器械识别跟踪、表面点云重建等功能，用户可根据业务需求结合 SDK 灵活的进行应用层开发。

1.2 环境要求

1.2.1 系统要求

Windows 10/11 系统

1.2.2 开发环境与编译语言

Visual Studio 2013/2015/2017/2019/2022，C++

2. SDK 接口说明

2.1 设备扫描接口

相关头文件 DeviceScan.h。

2.1.1 getDeviceInfo

原型

```
std::map<std::string, std::string> getDeviceInfo();
```

功能描述

获取可连接设备的 hostname 与 IP 地址。

参数

无。

返回值

std::map<std::string, std::string>类型：map 容器储存了所有可连接设备的 hostname 与 IP 地址，其中键为设备 hostname，值为设备 IP 地址。

2.1.2 updateDeviceInfo

原型

```
void updateDeviceInfo();
```

功能描述

刷新当前所有可连接设备的 hostname 与 IP 地址。

参数

无。

返回值

无。

2.2 设备连接接口

相关头文件 ARMDCombinedAPI.h。

2.2.1 connect

原型

```
int connect(std::string hostname/IP, bool autoExposure, bool errorPrint);
```

功能描述

连接与初始化设备。

参数

std::string 类型：设备对应的 hostname 或 IP；

bool 类型：设置连接时蜂鸣/不蜂鸣，默认蜂鸣；

bool 类型：设置自动曝光/固定曝光，默认自动曝光；

bool 类型：设置打印/不打印连接通讯错误信息，默认不打印。

返回值

int 类型：连接设备成功或失败状态值，其中-1 对应“连接失败”，0 对应“连接成功”。

2.2.2 disconnect

原型

```
void disconnect();
```

功能描述

断开与设备的连接。

参数

无。

返回值

无。

2.2.3 getConnectedDeviceInfo

原型

```
DeviceInfo getConnectedDeviceInfo();
```

功能描述

获取本地网卡与设备的 IP 地址。

参数

无。

返回值

DeviceInfo 类型：返回连接设备信息的结构体 DeviceInfo（详见 3.9 节），包含设备 hostname、IP 地址、MAC 地址、设备型号、固件版本号。

2.2.4 printAPIandFirmwareVersion

原型

```
std::vector<int> printAPIandFirmwareVersion();
```

功能描述

返回并打印 SDK 与固件版本号。

参数

无。

返回值

std::vector<int>类型：vector 容器前三位储存了 SDK 版本号，后三位储存了固件版本号。

2.3 器械设置接口

相关头文件 ARMDCombinedAPI.h。

2.3.1 generateAROM

原型

```
uint16_t generateAROM(std::string directory, ToolCalibrationData tool);
```

功能描述

根据用户输入的参数，生成器械标定文件（*.arom）。**注意**：该接口函数无法替代器械尖端/方向标定过程，若要确定器械尖端/方向坐标，请使用标定软件或调用 pivotTipCalibration/fixedTipCalibration/fixedDirCalibration 函数。

参数

std::string 类型：器械标定文件所在文件夹 directory；

ToolCalibrationData 类型：器械标定结构体 tool。

返回值

uint16_t 类型：生成器械标定文件（*.arom）成功或失败信息，其枚举类型命名空

间为 GenerationStatus。

2.3.2 convert2AROM

原型

uint16_t convert2AROM(std::string directoryOut, std::string directoryIn);

功能描述

将用户输入器械非标准标定文件（*.rom）转换为标准标定文件（*.arom）。

参数

std::string 类型：输出标准标定文件（*.arom）路径 directoryOut；

std::string 类型：输入非标准标定文件（*.rom）路径 directoryIn。

返回值

uint16_t 类型：生成器械标定文件（*.arom）成功或失败信息，其枚举类型命名空间为 GenerationStatus。

2.3.3 loadPassiveToolAROM

原型 1

void loadPassiveToolAROM(std::string directory, std::string parm);

功能描述

从指定文件夹目录读取所有被动反光器械标定文件（*.arom）。

参数

std::string 类型：被动反光器械标定文件所在文件夹 directory；

std::string 类型：导入方式（"cover"/"add"），其中"cover"表示覆盖已导入的器械标定数据，"add"表示在已导入的器械标定数据基础上添加，默认为覆盖。

返回值

无。

原型 2

void loadPassiveToolAROM(std::vector<std::string> directories, std::string parm);

功能描述

从指定文件夹目录读取所有被动反光器械标定文件（*.arom）。

参数

std::string 类型：被动反光器械标定文件所在文件夹 directory；

std::string 类型：导入方式（"cover"/"add"），其中"cover"表示覆盖已导入的器械标定数据，"add"表示在已导入的器械标定数据基础上添加，默认为覆盖。

返回值

无。

2.3.4 loadPassiveToolNAROM

原型 1

void loadPassiveToolNAROM(std::string dirctory, std::string parm);

功能描述

从指定文件夹目录读取所有器械非标准标定文件（*.rom）。**注意**：读取非标准标定文件存在风险，建议用户读取标准标定文件。

参数

std::string 类型：器械非标准标定文件所在文件夹 directory；

std::string 类型：导入方式（"cover"/"add"），其中"cover"表示覆盖已导入的器械标定数据，"add"表示在已导入的器械标定数据基础上添加，默认为覆盖。

返回值

无。

原型 2

void loadPassiveToolNAROM(std::vector<std::string> dirctories, std::string parm);

功能描述

从指定文件夹目录读取所有器械非标准标定文件（*.rom）。**注意**：读取非标准标定文件存在风险，建议用户读取标准标定文件。

参数

std::string 类型：器械非标准标定文件所在文件夹 directory；

std::string 类型：导入方式（"cover"/"add"），其中"cover"表示覆盖已导入的器械标定数据，"add"表示在已导入的器械标定数据基础上添加，默认为覆盖。

返回值

无。

2.3.5 loadActiveToolAROM

原型 1

void loadPassiveToolAROM(std::string directory, std::string parm);

功能描述

从指定文件夹目录读取所有主动发光器械标定文件（*.arom）。

参数

std::string 类型：主动发光器械标定文件所在文件夹 directory；

std::string 类型：导入方式（"cover"/"add"），其中"cover"表示覆盖已导入的器械标定数据，"add"表示在已导入的器械标定数据基础上添加，默认为覆盖。

返回值

无。

原型 2

```
void loadPassiveToolAROM(std::vector<std::string> directories, std::string parm);
```

功能描述

从指定文件夹目录读取所有主动发光器械标定文件（*.arom）。

参数

std::string 类型：主动发光器械标定文件所在文件夹 directory；

std::string 类型：导入方式（"cover"/"add"），其中"cover"表示覆盖已导入的器械标定数据，"add"表示在已导入的器械标定数据基础上添加，默认为覆盖。

返回值

无。

2.3.6 getToolStorage

原型

```
std::vector<ToolCalibrationData> getToolStorage();
```

功能描述

获取所有器械标定数据。

参数

无。

返回值

std::vector<ToolCalibrationData>类型：vector 容器，储存了每个器械的标定信息 ToolCalibrationData。

2.3.7 getTrackToolsNum

原型

```
int getTrackToolsNum();
```

功能描述

获取待跟踪器械数量。

参数

无。

返回值

int 类型：待跟踪器械数量，其与读入的器械标定文件（*.arom）数量一致。

2.3.8 clearToolStorage

原型

```
void clearToolStorage();
```

功能描述

清空所有跟踪器械。

参数

无。

返回值

无。

2.3.9 updateToolStorage

```
bool updateToolStorage(ToolCalibrationData tool);
```

功能描述

更新用于跟踪的器械标定数据。

参数

ToolCalibrationData 类型：器械新标定数据。

返回值

bool 类型：更新成功与否状态，若输入器械不属于当前用于跟踪的器械，则返回 false，更新失败。

2.3.10 getToolCalibrationData

```
ToolCalibrationData getToolCalibrationData(std::string name);
```

功能描述

获取指定已导入器械的标定数据。

参数

std::string 类型：器械名称。

返回值

ToolCalibrationData 类型：指定已导入器械的标定数据。

2.4 数据传输接口

相关头文件 ARMDCombinedAPI.h。

2.4.1 setTrackingDataTransmissionType

原型

```
void setTrackingDataTransmissionType(int type);
```

功能描述

设置传输跟踪数据的标志物类型。

参数

int 类型：标志物类型，其中 1 为只传输被动反光标志物跟踪数据（被动模式），2 为只传输主动发光标志物跟踪数据（主动模式），3 为混合不区分地发送被动反光与主动发光标志物跟踪数据（混合模式），4 为间隔区分地发送被动反光与主动发光标志物跟踪数据（交替模式）。**注意**：若不设置，瑞瞳设备默认只传输被动反光标志物跟踪数据，且在混合传输模式下无法区分主动反光与被动发光标志物跟踪数据，即在混合传输模式下，所有获取的标志物跟踪数据 MarkerPosition 的 type 值为 0（未知）。

返回值

无。

2.4.2 setTrackingDataTransformation

原型

```
bool setTrackingDataTransformation(double matrix[4][4]);
```

功能描述

设置器械跟踪数据的转换矩阵，该矩阵将左乘于器械的原始跟踪矩阵。

参数

double 二维数组类型：器械跟踪数据的转换矩阵。转换后的器械跟踪数据可从结构体 ToolTrackingData 中元素 transformM 获取（该元素类型为 Transformation）。**注意**：转换矩阵须为单位正交矩阵，否则输入后将被自动单位正交化。

返回值

设置成功与否，true 为成功，false 为失败，失败的情况包括：（1）输入矩阵的列向量间线性相关；（2）输入矩阵的列向量模长为 0。

2.4.3 startTracking

原型

```
void startTracking();
```

功能描述

开始跟踪器械。

参数

无。

返回值

无。

2.4.4 stopTracking

原型

```
void stopTracking();
```

功能描述

停止跟踪器械。

参数

无。

返回值

无。

2.4.5 startImaging

原型

```
void startImaging();
```

功能描述

开始传输双目图像数据。

参数

无。

返回值

无。

2.4.6 stopImaging

原型

```
void stopImaging();
```

功能描述

停止传输双目图像数据。

参数

无。

返回值

无。

2.4.7 startVideoMonitor

原型

```
void startVideoMonitor(bool r);
```

功能描述

开始传输可见光图像数据，图像默认为高分辨率尺寸 1920*1200。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

bool 类型：true 为高分辨率（1920*1200），false 为低分辨率（1280*720）。

返回值

无。

2.4.8 stopVideoMonitor

原型

```
void stopVideoMonitor();
```

功能描述

停止传输可见光图像数据。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

无。

2.4.9 setImagingMode

原型

```
void setImagingMode(int mode);
```

功能描述

设置传输双目红外图像类型。**注意**：该接口函数仅适用于开启双目红外图像传输后在交替模式下设置传输双目红外图像类型，在被动、主动以及混合模式下只能传输对应的一种图像类型，即包含被动反光标志物的图像（被动红外图像）、包含主动发光标志物的图像（主动红外图像）、包含被动反光与主动发光标志物的图像（主被动红外图像）。

参数

int 类型：0 为传输被动红外图像，1 为传输主动红外图像，在交替模式下默认传输被动红外图像。

返回值

无。

2.4.10 reconstructPointCloud

原型

void reconstructPointCloud(short width, short height, short leftX, short leftY, short rightX, short rightY);

功能描述

开始重建表面点云。**注意**：该接口函数仅适用于 PRO 型号。

参数

short 类型：重建区域长度 width（单位：像素）；

short 类型：重建区域宽度 height（单位：像素）；

short 类型：重建区域在左目图像 x 方向上的起始坐标 leftX；

short 类型：重建区域在左目图像 y 方向上的起始坐标 leftY；

short 类型：重建区域在右目图像 x 方向上的起始坐标 rightX；

short 类型：重建区域在右目图像 y 方向上的起始坐标 rightY。

返回值

无。

2.4.11 evaluateReconstruction

原型

bool evaluateReconstruction(std::vector<std::vector<float>> pc);

功能描述

重建点云数据质量评估。**注意**：该接口函数仅适用于 PRO 型号。

参数

std::vector<std::vector<float>>类型：待评估的点云数据。

返回值

点云质量，true 为质量好，false 为质量差。**注意**：当质量差时，需要进行再次重建，详细流程参照 5.5 节中示例。

2.4.12 setMaterial

原型

void setMaterial(int material);

功能描述

设置重建物体的材质。**注意**：当前仅支持硅胶（0）与石膏材质（1），默认为硅胶材质。该接口函数仅适用于 PRO 型号。

参数

int 类型：材质编号。

返回值

无。

2.4.13 trackingUpdate

原型

```
void trackingUpdate();
```

功能描述

刷新所有器械的跟踪数据与双目图像数据。

参数

无。

返回值

无。

2.4.14 monitoringUpdate

原型

```
void monitoringUpdate();
```

功能描述

刷新可见光图像数据。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

无。

2.4.15 getAllMarkers

原型

```
std::vector<MarkerPosition> getAllMarkers(int type);
```

功能描述

获取当前视场中指定类型所有标志物的跟踪数据。

参数

int 类型：标志物类型，其中 0 为所有标志物，1 为被动反光标志物，2 为主动发光

标志物。默认为所有标志物。

返回值

std::vector<MarkerPosition>类型：vector 容器储存了当前视场中指定类型所有标志物的跟踪数据 MarkerPosition。

2.4.16 getIFImageSize

原型

```
std::vector<int> getIFImageSize();
```

功能描述

获取双目相机图像尺寸。

参数

无。

返回值

std::vector<int>：vector 容器储存了双目相机图像长宽。

2.4.17 getVLImageSize

原型

```
std::vector<int> getVLImageSize();
```

功能描述

获取可见光相机图像尺寸。

参数

无。

返回值

std::vector<int>：vector 容器储存了可见光相机图像长宽。

2.4.18 setVLImageResolution

原型

```
void setVLImageResolution(bool level);
```

功能描述

设置可见光相机图像分辨率。

参数

bool 类型：false 为低分辨率（1280*720），true 为高分辨率（1920*1200）。

返回值

无。

2.4.19 getLeftImagingData

原型

char* getLeftImagingData();

功能描述

获取当前左目相机图像，图像尺寸为 1280*800 像素（SE、LITE、MAX 与 MAXV 型号）或 1920*1200 像素（PRO 型号）。

参数

无。

返回值

char*类型：当前左目相机图像数据。

2.4.20 getRightImagingData

原型

char* getRightImagingData();

功能描述

获取当前右目相机图像，图像尺寸为 1280*800 像素（SE、LITE、MAX 与 MAXV 型号）或 1920*1200 像素（PRO 型号）。

参数

无。

返回值

char*类型：当前右目相机图像数据。

2.4.21 getVideoMonitorData

原型

unsigned char* getVideoMonitorData();

功能描述

获取当前可见光相机图像，高分辨率图像尺寸为 1920*1200 像素，低分辨率图像尺寸为 1280*720 像素。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

无。

返回值

unsigned char*类型：当前可见光相机图像数据。

2.4.22 getReconstructionData

原型

```
std::vector<char*> getReconstructionData();
```

功能描述

获取用于重建表面点云的数据。**注意**：该接口函数仅适用于 PRO 型号。

参数

无。

返回值

std::vector<char*>类型：用于重建表面点云的数据。

2.4.23 getTrackingData

原型

```
std::vector<ToolTrackingData> getTrackingData(std::vector<MarkerPosition> cordi);
```

功能描述

获取当前所有器械的跟踪数据。

参数

std::vector<MarkerPosition>类型：vector 容器储存当前视场中所有标志物的跟踪数据 cordi。

返回值

std::vector<ToolTrackingData>类型：vector 容器储存了当前所有器械的跟踪数据 ToolTrackingData。

2.4.24 getUnMatchedMarkers

原型

```
std::vector<MarkerPosition> getUnMatchedMarkers();
```

功能描述

获取当前所有游离的未匹配标志物跟踪数据。该函数需在 getTrackingData 后运行，才可以获取正确数量。

参数

无。

返回值

std::vector<MarkerPosition>类型：vector 容器储存了当前视场中所有游离标志物的跟踪数据 MarkerPosition。

2.4.25 getSystemTime

原型

std::string getSystemTime();

功能描述

获取当前系统时间。

参数

无。

返回值

std::string 类型：当前系统时间。

2.4.26 getReconstructParameters

原型

std::vector<double> getReconstructParameters();

功能描述

获取用于表面点云重建的参数。**注意**：该接口函数仅适用于 PRO 型号。

参数

无。

返回值

std::vector<double>类型：用于表面点云重建的参数。

2.5 状态监测接口

相关头文件 ARMDCombinedAPI.h。

2.5.1 getGravityVector

原型

std::vector<double> getGravityVector();

功能描述

获取当前设备重力向量。

参数

无。

返回值

std::vector<double>类型：vector 容器储存了当前设备重力单位向量。

2.5.2 getConnectionStatus

原型

uint16_t getConnectionStatus();

功能描述

获取当前设备连接状态。

参数

无。

返回值

uint16_t 类型：当前设备连接状态，其枚举类型命名空间为 ConnectionStatus。

2.5.3 getTransmissionStatus

原型

uint16_t getTransmissionStatus();

功能描述

获取当前设备传输数据类型。

参数

无。

返回值

uint16_t 类型：当前设备传输数据类型，其枚举类型命名空间为 TransmissionStatus。

2.5.4 getReconstructionStatus

原型

uint16_t getReconstructionStatus();

功能描述

获取当前设备表面点云重建状态。

参数

无。

返回值

uint16_t 类型：当前设备点云重建状态，其枚举类型命名空间为 TransmissionStatus。

2.5.5 getSystemAlert

原型

uint16_t getSystemAlert();

功能描述

获取当前设备警告信息。

参数

无。

返回值

uint16_t 类型：当前设备温度、震动警告信息，其枚举类型命名空间为 DeviceAlert。

2.6 常用工具接口

相关头文件 ARMDCombinedAPI.h。

2.6.1 pivotTipCalibration

原型 1

uint16_t pivotTipCalibration(std::vector<std::vector<double>> rot, std::vector<std::vector<double>> tran, std::vector<double>& tip, double& error);

功能描述

根据器械绕尖端点转动的跟踪数据标定其在自身坐标系下的尖端坐标。

参数

std::vector<std::vector<double>>类型：跟踪数据的旋转矩阵 3n*3，其中 n 为跟踪数据的数量；

std::vector<std::vector<double>>类型：跟踪数据的平移向量 3n*1，其中 n 为跟踪数据的数量；

std::vector<double>&类型：用于存储标定结果，即器械在其自身坐标系下的尖端坐标，若标定失败，返回空白 vector 容器；

double&类型：用于存储标定误差，若标定失败，返回-1。

返回值

uint16_t 类型：标定成功与否状态，其枚举类型命名空间为 CalibrationAlert。

原型 2

uint16_t pivotTipCalibration(ToolCalibrationData& tool, std::vector<ToolTrackingData> trackingdataVect, std::vector<double>& tip, std::vector<double>& offset, double& error, bool update = false);

功能描述

根据器械绕尖端点转动的跟踪数据标定其尖端。

参数

ToolCalibrationData&类型：待标定的器械数据；

std::vector<ToolTrackingData>类型：器械转动过程中采集的跟踪数据；

std::vector<double>&类型：用于存储标定结果，即器械在其自身坐标系下的尖端坐标，若标定失败，返回空白 vector 容器；

std::vector<double>&类型：用于存储当前标定与上次标定结果间在器械自身坐标系下 x，y，z 方向上的偏差，若标定失败，返回空白 vector 容器；

double&类型：用于存储标定误差，若标定失败，返回-1；

bool 类型：是否更新输入待标定的器械数据中尖端坐标信息，默认为 false。

返回值

uint16_t 类型：标定成功与否状态，其枚举类型命名空间为 CalibrationAlert。

2.6.2 fixedTipCalibration

uint16_t fixedTipCalibration(ToolCalibrationData& tool, ToolTrackingData trackingdata, MarkerPosition point, std::vector<double>& tip, std::vector<double>& offset, bool originalTip = false, bool update = false);

功能描述

根据器械的当前跟踪数据与瑞瞳坐标系下尖端点坐标标定其在自身坐标系下的尖端坐标。

参数

ToolCalibrationData&类型：待标定的器械数据；

ToolTrackingData 类型：器械当前跟踪数据；

MarkerPosition 类型：瑞瞳坐标系下尖端点坐标；

std::vector<double>&类型：用于存储标定结果，即器械在其自身坐标系下的尖端坐标，若标定失败，返回空白 vector 容器；

std::vector<double>&类型：用于存储当前标定与上次标定结果间在器械自身坐标系下 x，y，z 方向上的偏差，若标定失败，返回空白 vector 容器；

bool 类型：是否将标定的尖端设置为原点，默认为 false；

bool 类型：是否更新输入待标定的器械数据中尖端坐标信息，默认为 false。

返回值

uint16_t 类型：标定成功与否状态，其枚举类型命名空间为 CalibrationAlert。

2.6.3 fixedDirCalibration

uint16_t fixedDirCalibration(ToolCalibrationData& tool, ToolTrackingData trackingdata, double vector[3], std::vector<double>& dir, bool normalize = false, bool update = false);

功能描述

根据器械的当前跟踪数据与瑞瞳坐标系下方向向量标定其在自身坐标系下的方向向量。

参数

ToolCalibrationData&类型：待标定的器械数据；

ToolTrackingData 类型：器械当前跟踪数据；

double[3]类型：瑞瞳坐标系下方向向量；

std::vector<double>&类型：用于存储标定结果，即器械在其自身坐标系下的方向向量，若标定失败，返回空白 vector 容器；

bool 类型：是否将标定的方向向量归一化，默认为 false；

bool 类型：是否更新输入待标定的器械数据中方向向量信息，默认为 false。

返回值

uint16_t 类型：标定成功与否状态，其枚举类型命名空间为 CalibrationAlert。

2.6.4 setIFLuminance

原型

```
void setIFLuminance(double l, int side = 0);
```

功能描述

设置双目红外图像显示的亮度。

参数

double 类型：亮度，范围为[-1, 1]，1 表示最亮，-1 表示最暗，若输入超过 1 则为 1，若小于-1 则为-1；

int 类型：指定相机，0 为双目红外相机，1 为左目红外相机，2 为右目红外相机，默认为 0。

返回值

无。

2.6.5 setIFContrast

原型

```
void setIFContrast(double c, int side = 0);
```

功能描述

设置双目红外图像显示的对比度。

参数

double 类型：对比度，范围为[-1, 1]，1 表示对比最强，-1 表示对比最弱，若输入

超过 1 则为 1，若小于-1 则为-1；

int 类型：指定相机，0 为双目红外相机，1 为左目红外相机，2 为右目红外相机，默认为 0。

返回值

无。

2.6.6 setAreaDisplay

原型

void setAreaDisplay(short width, short height, short leftX, short leftY, short rightX, short rightY);

功能描述

在双目红外图像上绘制矩形框。

参数

short 类型：绘制矩形区域长度 width（单位：像素）；

short 类型：绘制矩形区域宽度 height（单位：像素）；

short 类型：绘制矩形区域在左目图像 x 方向上的起始坐标 leftX；

short 类型：绘制矩形区域在左目图像 y 方向上的起始坐标 leftY；

short 类型：绘制矩形区域在右目图像 x 方向上的起始坐标 rightX；

short 类型：绘制矩形区域在右目图像 y 方向上的起始坐标 rightY。

返回值

无。

2.6.7 setAreaHidden

原型

void setAreaHidden();

功能描述

隐藏在双目红外图像上绘制的矩形框。

参数

无。

返回值

无。

2.6.8 trackingDataProjection

原型

ToolTrackingData trackingDataProjection(ToolTrackingData targetTool, ToolTrackingData sourceTool);

功能描述

将一个器械（源器械）的跟踪数据投影至另一个器械（目标器械）坐标系下。

参数

ToolTrackingData 类型：目标器械跟踪数据 targetTool；

ToolTrackingData 类型：源器械跟踪数据 sourceTool。

返回值

ToolTrackingData 类型：源器械在目标器械坐标系下的跟踪数据。

2.6.9 setReferenceTool

原型

bool setReferenceTool(std::string toolName);

功能描述

将一个器械设置为参考器械，则 getTrackingData()返回的当前所有器械的跟踪数据中第一个数据为参考器械数据，且通过调用其它器械跟踪数据（ToolTrackingData）的 projected()可以获得该器械在参考器械坐标系下的跟踪数据。

参数

std::string 类型：参考器械名称。

返回值

bool 类型：设置成功与否状态，true（工具集里面存在该器械，设置成功），false（工具集里面不存在该器械，设置失败）。

2.6.10 getTrackingVolume

原型

std::vector<int> getTrackingVolume();

功能描述

获取当前设备跟踪范围。

参数

无。

返回值

std::vector<int>类型：容器内储存设备的近（Nx, Ny, Nz）、中（Mx, My, Mz）、远（Fx, Fy, Fz）三个跟踪平面的尺寸，整个跟踪范围为相邻平面组成的棱台，则在这三个平面

上的跟踪范围为{[-Nx/2, Nx/2], [-Ny/2, Ny/2], [Nz, Nz]}、{[-Mx/2, Mx/2], [-My/2, My/2], [Mz, Mz]}、{[-Fx/2, Fx/2], [-Fy/2, Fy/2], [Fz, Fz]}，若未连接设备，则返回空容器。

2.6.11 markerInOutTrackingVolume

原型

```
bool markerInOutTrackingVolume(MarkerPosition p);
```

功能描述

判断输入的跟踪标志物坐标是否在跟踪范围内。

参数

MarkerPosition 类型：输入的跟踪标志物。

返回值

bool 类型：如在跟踪范围内，返回 true，反之，返回 false。

2.6.12 beep

原型

```
void beep(int delay);
```

功能描述

开始蜂鸣，且将在设定延迟时间后关闭。

参数

int 类型：延迟时间，单位：毫秒。

返回值

无。

2.6.13 laser

原型

```
void laser(bool status);
```

功能描述

开启/关闭定位激光。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

bool 类型：开启（true），关闭（false）。

返回值

无。

2.6.14 laserOn

原型

void laserOn(int delay);

功能描述

打开定位激光，且激光将在设定延迟时间后关闭。**注意**：该接口函数仅适用于 PRO 与 MAXV 型号。

参数

int 类型：延迟时间，单位：毫秒。

返回值

无。

2.6.15 getNetAdaptorInfo

原型

NetAdaptorInfo getNetAdaptorInfo();

功能描述

获取与设备连接的网卡信息。

参数

无。

返回值

NetAdaptorInfo 类型：网卡信息，包括网卡的名称、连接类型、网速、连接状态，其中，网速单位：MB/s，详见 3.8 节。

2.6.16 setDeviceConnectionMode

原型

bool setDeviceConnectionMode(bool autom, std::string IP = “”, std::string MK = “”, std::string GW = “”);

功能描述

设置设备连接模式，固定 IP 或自动 IP，设备默认为自动 IP。

参数

bool 类型：设置为自动 IP 模式（true）或固定 IP 模式（false）；

std::string 类型：设备 IP，当设置固定 IP 模式时起作用，默认为“”；

std::string 类型：设备子网掩码，当设置固定 IP 模式时起作用，默认为“”；

std::string 类型：设备网关，当设置固定 IP 模式时起作用，默认为“”。

返回值

bool 类型：设置成功（true）或失败（false）。

2.6.17 getLog

原型

```
std::vector<std::string> getLog();
```

功能描述

获取日志信息，包含设备连接、断开、中断、设备警报以及跟踪数据传输开关、图像（双目红外、可间光）数据传输开关、点云重建等时间记录。

参数

无。

返回值

std::vector<std::string>类型：日志信息。

2.6.18 saveLog

原型

```
bool saveLog(std::string path);
```

功能描述

将日志信息保存到指定路径。

参数

std::string 类型：保存路径。

返回值

bool 类型：保存成功（true）或失败（false）。

2.7 点云重建接口

相关头文件 ReconstructPointCloud.h。**注意**：该类接口函数仅适用于 PRO 型号，且由于涉及 OpenCV 库，调用该类接口函数时需要引用相应的 openCV 库文件，若仅使用瑞瞳设备的跟踪定位功能，可以不引用该头文件与依赖库文件。

2.7.1 initializeParameters

原型

```
void initializeParameters(std::vector<double> infor);
```

功能描述

初始化点云重建参数。

参数

std::vector<double>类型：点云重建参数，可通过函数 getReconstructParameters()获

取；

返回值

无。

2.7.2 setReconstructData

原型

```
void setReconstructData(std::vector<std::vector<char>> data);
```

功能描述

输入表面点云重建所需的数据。

参数

std::vector<std::vector<char>>类型：表面点云重建所需的数据。

返回值

无。

2.7.3 getPointCloud

原型

```
PointCloudInfo  getPointCloud(bool texture);
```

功能描述

获取重建的表面点云。

参数

bool 类型：是否添加纹理。

返回值

PointCloudInfo 类型：重建的表面点云。

2.7.4 getTextureImageMat

原型

```
cv::Mat  getTextureImageMat();
```

功能描述

获取重建点云纹理图，图片大小为 1920*1200，其格式为 openCV 的 Mat 类型。

参数

无。

返回值

cv::Mat 类型：点云纹理图。

2.7.5 getTextureImageUChar

原型

```
unsigned char*  getTextureImageUChar();
```

功能描述

获取重建点云纹理图，图片大小为 1920*1200，其格式为 unsigned char*类型。

参数

无。

返回值

unsigned char*类型：点云纹理图。

2.7.6 getDepthImage

原型

```
cv::Mat  getDepthImage();
```

功能描述

获取重建点云的深度图，其格式为 openCV 的 Mat 类型。

参数

无。

返回值

cv::Mat 类型：重建点云的深度图。

2.7.7 getDepthImageSize

原型

```
std::vector<int>  getDepthImageSize();
```

功能描述

获取重建点云深度图的尺寸。

参数

无。

返回值

std::vector<int>类型：重建点云深度图的宽、高。**注意**：深度图为三通道图像，每个通道分别为 x，y，z 方向的深度。

2.7.8 setDefaultColor

原型

```
void  setDefaultColor(int r, int g, int b, int a);
```

功能描述

为无纹理的点设置默认颜色与透明度，其值范围为 0-255，若不设置，默认值分别为 0,0,0,255。

参数

int 类型：颜色 r 值，范围为 0-255；

int 类型：颜色 g 值，范围为 0-255；

int 类型：颜色 b 值，范围为 0-255；

int 类型：透明度值，范围为 0-255。

返回值

无。

2.7.9 project2DImagePosition

原型

```
std::vector<int> project2DImagePosition(int x, int y);
```

功能描述

获取纹理图的（二维）点对应点云上的（三维）点。

参数

int 类型：纹理图点坐标 x 值，范围为 0-(1920-1)的整数；

int 类型：纹理图点坐标 y 值，范围为 0-(1200-1)的整数。

返回值

std::vector<int>类型：容器储存对应的点云中点编号。

2.7.10 project2DImageArea

原型

```
std::vector<int> project2DImageArea(std::vector<std::pair<int,int>> area);
```

功能描述

获取纹理图的（二维）点集合对应点云上的（三维）点集合。

参数

std::vector<std::pair<int,int>>类型：纹理图点坐标 x 与 y 值的容器，x 与 y 值的范围分别为 0-(1920-1)与 0-(1200-1)。

返回值

std::vector<int>类型：容器储存对应的点云中点编号。

2.7.11 project2DImageRectArea

原型

```
std::vector<int> project2DImageRectArea(int cx, int cy, int w, int h);
```

功能描述

获取纹理图的矩形区域内（二维）点集合对应点云上的（三维）点集合。

参数

int 类型：矩形中心点坐标 x，范围为 0-(1920-1)；

int 类型：矩形中心点坐标 y，范围为 0-(1200-1)；

int 类型：矩形宽 w；

int 类型：矩形高 h。

返回值

std::vector<int>类型：容器储存对应的点云中点编号。

2.7.12 project2DImageCircleArea

原型

```
std::vector<int> project2DImageCircleArea(int cx, int cy, int r);
```

功能描述

获取纹理图的圆形区域内（二维）点集合对应点云上的（三维）点集合。

参数

int 类型：圆形中心点坐标 x，范围为 0-(1920-1)；

int 类型：圆形中心点坐标 y，范围为 0-(1200-1)；

int 类型：圆形半径 r。

返回值

std::vector<int>类型：容器储存对应的点云中点编号。

2.7.13 savePLY

原型 1

```
bool savePLY(std::string filename, std::vector<std::vector<float>> cloud);
```

功能描述

保存重建的表面点云。

参数

std::string 类型：保存路径；

std::vector<std::vector<float>>类型：重建的表面点云，其中每个点坐标的数据类型为 std::vector<float>。

返回值

bool 类型：保存成功与否状态。

原型 2

bool savePLY(std::string filename, std::vector<cv::Point3f> cloud);

功能描述

保存重建的表面点云。

参数

std::string 类型：保存路径；

std::vector<std::vector< cv::Point3f >>类型：重建的表面点云，其中每个点坐标的数据类型为 cv::Point3f。

返回值

bool 类型：保存成功与否状态。

2.8 增强现实融合显示接口

相关头文件 ARprojection.h。**注意**：该类接口函数仅适用于 PRO 型号，且由于涉及 OpenCV 库，调用该类接口函数时需要引用相应的 openCV 库文件，若仅使用瑞瞳设备的跟踪定位功能，可以不引用该头文件与依赖库文件。

2.8.1 initialization

原型

bool initialization(std::vector<double> infor, bool r = true);

功能描述

初始化增强现实参数。

参数

std::vector<double>类型：增强现实融合参数，可通过函数 getReconstructParameters()获取；

bool 类型：融合的可见光图像分辨率，其中 true 为高分辨率，false 为低分辨率，默认为高分辨率。

返回值

bool 类型：初始化成功与否状态。

2.8.2 getARprojection

原型

std::vector<cv::Point3d> getARprojection(std::vector<cv::Point3d> orgPointCloud,

```
double trackingMatrix[4][4]);
```

功能描述

将虚拟模型投影至可见光图像。

参数

std::vector<cv::Point3d>类型：虚拟模型点坐标；

double 数组 4*4：虚拟模型在瑞瞳坐标系下跟踪矩阵。

返回值

std::vector<cv::Point3d>类型：虚拟模型点在可见光图像坐标系下坐标。

3. SDK 重要数据结构

3.1 器械标定信息数据结构 ToolCalibrationData

```
struct ToolCalibrationData
{
    std::string name = “”;                  //器械名称
    bool markerType = false;                //器械类型：false（被动），true（主动）
    int minNumMarker = 3;                   //最少匹配标志物数量
    int planeNum = 1;                       //平面数量
    std::vector<MarkerPlane> planes;        //每个平面标志物
    std::vector<MarkerPosition> points;     //虚拟点
    double dir[3] = {0.0, 0.0, 0.0};        //虚拟方向
    double pin[3] = {0.0, 0.0, 0.0};        //尖端坐标
    double calbError = -1;                  //标定误差，-1 表示器械未标定
    double maxFRE = 1;                      //最大匹配误差
    double maxAngle = 60;                   //最大匹配角度
    int algorithmType = 1;                  //匹配算法：0 平衡，1 精度优先，2 速度优先
    int structureType = 0;                  //器械结构：0 单平面，1 多平面，2 非平面
};
```

3.2 跟踪信息数据结构 ToolTrackingData

```
struct ToolTrackingData
{
    std::string name = “”;                  //名称
    std::string timespec = “”;              //当前时间
    Transformation transform;               //位姿信息
    int matchStatus = 0;                    //匹配状态：1（跟踪），0（丢失）
    int matchedPlane = 0;                   //匹配平面编号
    int matchedMarkersNum = 0;              //平面内匹配标志物数量
    MarkerPlane plane;                      //标志物坐标
    std::vector<MarkerPosition> points;     //虚拟点坐标
    double dir[3] = {0.0, 0.0, 0.0};        //虚拟方向
```

```
    double error;                          //匹配误差

    ToolTrackingData projected();          //将当前器械跟踪数据映射至参考工具空间
}
```

3.3 标志物数据结构 MarkerPosition

```
struct MarkerPosition
{
    double P[4] = {0.0, 0.0, 0.0, 1};                  //标志物坐标
    double dir[3] = {1.0, 0.0, 0.0};                   //标志物方向
    int leftExposure = 0;                              //标志物亮度
    int rightExposure = 0;                             //标志物亮度
    int type = 0;                                      //标志物类型：0（未知），1（被动），2（主动）
    bool RF = true;                                    //标志物识别结果：true（真实），false（疑似）

    MarkerPosition operator*(double matrix[4][4]);     //乘法，返回矩阵 matrix 左乘后位置
};
```

3.4 器械平面数据结构 MarkerPlane

```
struct MarkerPlane
{
    int index = 0;                                     //平面编号 0-(器械平面数量-1)
    std::vector<MarkerPosition> markers;               //平面上标志物信息
    double dir[3] = {0.0, 0.0, 0.0};                   //平面法向量
};
```

3.5 跟踪位姿信息数据结构 Transformation

```
struct Transformation
{
    uint16_t status;              //匹配状态，其枚举类型命名空间为 TransformationStatus
    double matrix[4][4];          //姿态矩阵
    double qw, qx, qy, qz;        //姿态四元数
    double tx, ty, tz;            //器械尖端坐标
    double yaw, pitch, roll;      //器械旋转角度

    Transformation inversed();    //计算姿态逆矩阵
    Transformation transformed();  //根据设置的转换矩阵，计算转换后姿态矩阵
};
```

3.6 点数据结构 PointInfo

```
struct PointInfo
{
    float x, y, z;                //点在瑞瞳空间坐标
    bool textured;                //是否具有纹理
    unsigned char r, g, b, a;     //纹理与透明度，其范围为 0-255 的整数
    int p, q;                     //纹理图对应坐标
};
```

3.7 点云数据结构 PointCloudInfo

```
struct PointCloudInfo
{
    std::vector<PointInfo> points;                    //点云中所有点
    int num = 0;                                      //点云中所有点数量
    int stage = -1;                                   //点云对应的生成参数，重建后自动生成

    PointInfo getPoint(int index);                    //获取指定编号的点信息
    PointCloudInfo getPoints(std::vector<int> index); //获取指定编号集合的点信息集合
    PointCloudInfo getTexturedPoints();               //获取所有具有纹理的点组成的点云
    PointCloudInfo getUntexturedPoints();             //获取所有无纹理的点组成的点云
    PointCloudInfo transformed(double mat[4][4]);     //变换点云，变换矩阵为 double 类型的 4*4 数组
    void removeUntexturedPoints();                    //删除源点云中无纹理的点，并更新
    void insertPoint(PointInfo point);                //在源点云中插入新点，并更新
    void insertPoints(std::vector<PointInfo> points); //在源点云中插入新点集合，并更新
    void concatenate(PointCloudInfo pc);              //将输入点云与源点云拼接，并更新
    std::vector<float> getBoundingBox();              //获取包围盒{maxX, minX, maxY, minY, maxZ, minZ}
    std::vector<float> getBoundCenter();              //获取点云包围盒中心点坐标
    std::vector<float> getGeometricalCenter();        //获取点云的几何中心坐标
};
```

3.8 网卡信息数据结构 NetAdaptorInfo

```
struct NetAdaptorInfo
{
    std::string name = “”;                            //设备连接的网卡名称
    std::string connectionType = “”;                  //设备连接的网卡连接类型
    int speed = 0;                                    //设备连接的网卡网速
    bool linked = false;                              //设备连接的网卡连接状态
};
```

3.9 连接设备信息数据结构 DeviceInfo

```
struct DeviceInfo
{
    std::string hostname = “”;                        //连接设备的 hostname
    std::string IP = “”;                              //连接设备的 IP 地址
    std::string MAC = “”;                             //连接设备的 MAC 地址
    int deviceType = DeviceType::None;                //连接设备的型号
    int firmwareVersion[3] = { 0, 0, 0 };             //连接设备的固件版本号
};
```

4. SDK 重要枚举类型的命名空间

4.1 设备类型 DeviceType

```
namespace DeviceType
{
```

```
        enum value
        {
            NONE = 0x0000,                  //设备型号未知
            SE = 0x0001,                    //SE型号
            LITE = 0x0002,                  //LITE型号
            MAX = 0x0003,                   //MAX型号
            MAXV = 0x0004,                  //MAXV型号
            PRO = 0x0005,                   //PRO型号
            TD = 0x0006,                    //TD型号
        };
    }
```

4.2 器械标定文件生成状态 GenerationStatus

```
    namespace GenerationStatus
    {
        enum value
        {
            OK = 0x0000,                        //生成成功
            MinMatchedNumLess = 0x0001,         //生成失败，最少匹配标志物数量过少
            MinMatchedNumMore = 0x0002,         //生成失败，最少匹配标志物数量过多
            PlaneNumLess = 0x0003,              //生成失败，平面数量过少
            PlaneNumMore = 0x0004,              //生成失败，平面数量过多
            PlaneNumNotMatching = 0x0005,       //生成失败，平面数量不匹配
            MarkerNumLess = 0x0006,             //生成失败，平面内标志物数量过少
            MarkerNumMore = 0x0007,             //生成失败，平面内标志物数量过多
            MarkerNumLimitExceed = 0x0008,      //生成失败，器械标志物数量过多
            DistanceTooSmall = 0x0009,          //生成失败，标志物间距过小
            NameError = 0x000A,                 //生成失败，文件名包含中文与符号
            PathNotExist = 0x000B,              //生成失败，文件存储路径不存在
            TypeError = 0x000C,                 //生成失败，文件类型错误
        };
    }
```

4.3 设备传输数据类型 TransmissionStatus

```
    namespace TransmissionStatus
    {
        enum value
        {
            NoneData = 0x0000,                  //无数据传输
            TrackingData = 0x0001,              //跟踪数据传输
            ImagingData = 0x0002,               //双目红外图像据传输
            AllData = 0x0003,                   //跟踪+双目图像数据传输
            Suspend = 0x0004,                   //数据传输暂停
            ReconstructionReady = 0x0005,       //表面点云数据传输完毕
            LMonitor = 0x0006,                  //低分辨率可见光图像传输
```

```
            HMonitor = 0x0007,                    //高分辨率可见光图像传输
        };
    }
```

4.4 设备跟踪数据传输类型 TransmissionType

```
    namespace TransmissionType
    {
        enum value
        {
            None = 0x0000,                        //默认数据传输类型
            Passive = 0x0001,                     //被动反光数据传输类型
            Active = 0x0002,                      //主动发光数据传输类型
            Mixed = 0x0003,                       //混合数据传输类型
            Alternate = 0x0004,                   //交替数据传输类型
        };
    }
```

4.5 设备连接状态 ConnectionStatus

```
    namespace ConnectionStatus
    {
        enum value
        {
            Connected = 0x0000,                   //连接成功
            DisConnected = 0x0001,                //断开成功
            DisConnectedFailed = 0x0002,          //断开失败
            PortConflicts = 0x0003,               //连接失败，连接端口冲突
            ResloveError = 0x0004,                //连接失败，IP解析错误
            InitialUDPError = 0x0005,             //连接失败，初始化错误
            Interruption = 0x0006,                //连接中断
            InvalidAddr = 0x0007,                 //连接失败，IP地址无效
            DeviceInexist = 0x0008,               //连接失败，设备不存在
            TimeOut = 0x0009,                     //连接失败，设备连接超时
            UnknownError = 0x000A,                //连接失败，未知错误
        };
    }
```

4.6 设备警告信息 DeviceAlert

```
    namespace DeviceAlert
    {
        enum value
        {
            Normal = 0x0000,            //无警告
            Vibration = 0x0001,         //震动警告，剧烈震动已影响设备精度，建议返厂
            TempTooHigh = 0x0002,       //高温警告，高温已影响设备精度，请为设备降温
            TempTooLow = 0x0003,        //低温警告，低温已影响设备精度，请提升环境温度
        };
```

```
}
```

4.7 器械标定状态 CalibrationAlert

```
namespace CalibrationAlert
{
    enum value
    {
        Normal = 0x0000,                    //标定成功
        AbnormalData = 0x0001,              //标定失败，输入的器械跟踪数据异常
        InadequateData = 0x0002,            //标定失败，输入的器械跟踪数据不足
        ComputationError = 0x0003,          //标定失败，标定过程计算异常
    };
}
```

4.8 器械跟踪状态 TransformationStatus

```
namespace TransformationStatus
{
    enum values
    {
        Enabled = 0x0000,                   //器械跟踪成功
        PartiallyOutOfVolume = 0x0001,      //器械跟踪成功，但部分标志物超出视场范围
        OutOfVolume = 0x0002,               //器械跟踪成功，但全部标志物超出视场范围
        PartiallyMatched = 0x0003,          //器械跟踪成功，但部分标志物未识别到
        TooFewMarkers = 0x0004,             //器械跟踪失败，匹配标志物数量低于设定值
        BadTransformFit = 0x0005,           //器械跟踪失败，器械跟踪误差超过设定值
        TooLargeAngle = 0x0006,             //器械跟踪失败，器械跟踪角度超过设定值
        ToolMissing = 0x0009,               //器械跟踪失败，视场中不存在器械的标志物
    };
}
```

5. Demo 使用指南

测试该 Demo 的系统为 Window 10，开发环境为 Visual Studio 2013/2015/2017/2019/2022。提供的 Demo 包含同步刷新器械跟踪数据示例（SynchronousDemo）与异步刷新器械跟踪数据示例（AsynchronousDemo），其中同步刷新器械跟踪数据，指的是在同一线程顺序执行**标志物跟踪数据传输**与**器械跟踪数据更新**；异步刷新器械跟踪数据，指的是在不同线程分别执行**标志物跟踪数据传输**与**器械跟踪数据更新**。由于标志物跟踪数据传输频率较高（60HZ、120HZ），若同步刷新器械跟踪数据，当识别到的标志物数量较多（>100 个）或待跟踪的器械数量较多（>10 个）时，会延迟标志物跟踪数据的传输，应采用异步刷新器械跟踪数据。

5.1 连接准备

接入瑞瞳设备，确认设备背面标记的 hostname，例如 RT-SE000001.local。

5.2 文件目录

include 文件夹包含 SDK 头文件（*.h），lib 文件夹包含 SDK 库文件（ARMDstereotracking.lib 与 ARMDstereoreconstruction.lib 或 ARMDstereotrackingd.lib 与 ARMDstereoreconstructiond.lib），tool 文件夹包含器械标定文件（*.arom），调用 SDK 的示例代码为 AsynchronousDemo.cpp 与 SynchronousDemo.cpp。

5.3 使用流程

使用 Visual Studio 打开文件夹 SynchronousDemo 下面的*.sln 工程文件，导入 SynchronousDemo 与 AsynchronousDemo 两个项目，其中项目 SynchronousDemo 为同步刷新器械跟踪数据示例，项目 AsynchronousDemo 为异步刷新器械跟踪数据示例。

5.4 工程配置

对于两个项目，分别设置对应的**包含目录**（include 文件夹）与**库目录**（lib/Debug 或 lib/Release 文件夹）路径，**链接器→输入→附加依赖项**添加库文件，**C/C++→预处理器→预处理器定义**添加 ARMDCOMBINEDAPI_EXPORTS。

5.5 调用流程

Step1：生成器械标定文件（*.arom），调用 generateAROM()

在 tool 路径下，根据输入的名称、器械标志物坐标、最少匹配标志物数量，生成标定文件。该步骤非必须，若已有器械标定文件，可跳过此步骤。

Step2：自动/手动连接设备并初始化，调用 autoConnect()/connect(hostname)

对于手动连接，须设置设备的 hostname；对于自动连接，自动扫描局域网内可连接设备，并连接找到的第一个设备。

Step3：读入器械标定文件，调用 configureTools()

读入 tool 路径下所有器械标定文件（*.arom）。

Step4-1：显示双目相机图像与器械/标志物跟踪信息，调用 printTrackingImagingData()

双目相机图像与器械/标志物跟踪数据，其中器械跟踪信息包括名称、标志物坐标、当前时间、配准误差、姿态矩阵和尖端点坐标；标志物跟踪信息包括视场中所有标志物坐标、未匹配标志物坐标。

Step4-2：重建表面点云，调用 reconstructPointCloud()

设置重建区域，采集重建数据完毕后，生成区域内物体表面点云并保存 ply 格式数据。

Step5：关闭系统，调用 closeSystem()

停止跟踪与图像数据传输，断开连接。

6. 常见问题

Q：启动系统时显示连接失败。

A：检查连接时输入的 hostname 是否与设备背面标记的一致。

Q：手动连接设备成功，但自动连接设备失败。

A：请关闭**所有**防火墙后，重新自动连接。